In [ ]:
```
int

float

str

bool

bin  oct  hex  complex
```

In [ ]:
```
it is very very important to convert one data type to another data type

type casting

I want to convert integer value to float
I want to convert integer value to str
integer to bool
integer to complex


float to int
float to bool
float to str
float to complex


str to int
str to float
str to bool
str to complex

bool to int
bool to float
bool to str
bool to complex

complex to int
complex to float
complex to str
complex to bool


- If it is possible or not

- If it is possibel how can we convert

- If its is not possible what is the error
```

$Case - 1$

int to other data types

In [1]:
```
number=100
type(number)
```

Out[1]:
```
int
```

In [9]:
```
# int to float
number=100
number_type=type(number) # int
print(number_type)

number_float=float(number) # float(100)=100.0
print(type(number_float))   # float
print(number_float)  # 100.0
```

```
<class 'int'>
<class 'float'>
100.0
```

In [10]:
```
# int to str
number=100
str(number)   # '100'
```

Out[10]:
```
'100'
```

In [11]:
```
# int to bool
number=100   # int
bool(number)
```

Out[11]:
```
True
```

In [12]:
```
number=-100
bool(number)
```

Out[12]:
```
True
```

In [13]:
```
number=0
bool(number)
```

Out[13]:
```
False
```

- bool conversion on postive or negative integer numbers is True
- bool conversion on zero is False

In [14]:
```
number=100
complex(number)  # complex(100)

# 100 is real value
```

Out[14]:
```
(100+0j)
```

In [16]:
```
#complex(<real>,<imag>)
complex(100)

# real + imaJ  a+bj
# 100+0j
```

Out[16]:
```
(100+0j)
```

In [17]:
```
complex(100,200)  # 100 real   200 imag
```

Out[17]:
```
(100+200j)
```

In [21]:
```
import random
random.randint(10,20)   # take cursor inside bracket shift+tab
```

Out[21]:
```
17
```

In [22]:
```
complex(100)
```

Out[22]:
```
(100+0j)
```

- first it is method or not
- method means it is a function
- function require brackets
- inside the bracket , do you need provide any values
- Inside bracket yes i need to provide some values
- No, Inside bracket no need to provide values

In [23]:
```
complex(100) # 100+0j
complex(100,200) # 100+200j
complex() # 0j
```

Out[23]:
```
0j
```

In [29]:
```
random.randint
```

Out[29]:
```
<bound method Random.randint of <random.Random object at 0x0000014E712D5EA
0>>
```

- Name error : Name not defined
  - Check above lines have you provided any variable with that name
- syntax error
- Attribute error:
  - Father does not have a kid with that name
  - Random does not have a attribute with randin
  - check dir(random)
  - check all the attributes

In [30]:
```
number=100
float(number) # 100.0
str(number)   # '100'
bool(number)  # True
complex(number) # 100+0j
```

Out[30]:
```
(100+0j)
```

In [32]:
```
number=100.5  # float
print(int(number))  # 100
print(str(number))   # '100.5' (hold)
print(bool(number))  # True (hold)
print(complex(number)) # 100.5+0j
```

```
100
100.5
True
(100.5+0j)
```

In [33]:
```
100e3  # 100*1000=100000.0
```

Out[33]:
```
100000.0
```

In [34]:
```
10e2 # 10*100=1000.0
```

Out[34]:
```
1000.0
```

In [35]:
```
10e4 # 10* 10000=100000.0
```

Out[35]:
```
100000.0
```

In [37]:
```
100e-3   # 100/1000= 0.1
```

Out[37]:
```
0.1
```

In [39]:
```
10e-4  # 10/10000
```

Out[39]:
```
0.001
```

- e4 means multiply with 4 zeros combination(10000)
- e+4 means e4 only
- e-4 mean divide with 4 zeros combination

In [ ]:
```
1.0000000000e-8  # 1/100000000= 0.000000001

# use case ===== > 0
```

In [40]:
```
123e3   # 123*1000=123000
```

Out[40]:
```
123000.0
```

In [41]:
```
number="100" # string
print(int(number))     # 100
print(float(number))   # 100.0
print(bool(number))    # True
print(complex(number)) # 100+0j
```

```
100
100.0
True
(100+0j)
```

In [43]:
```
number="100.5" # string
#print(int(number))     # fail
print(float(number))   # 100.5
print(bool(number))    # True
print(complex(number)) # 100.5+0j
```

```
100.5
True
(100.5+0j)
```

In [ ]:
```
int("100")  # works
int('100.5') # fail
```

In [47]:
```
number="python" # string
#print(int(number))     # error
#print(float(number))   # error
print(bool(number))    # True
#print(complex(number)) # error
```

```
True
```

In [48]:
```
number="0" # string
print(int(number))     # 0
print(float(number))   # 0.0
print(bool(number))    # True
print(complex(number)) # 0j
```

```
0
0.0
True
0j
```

In [49]:
```
number="$" # string
#print(int(number))     # error
#print(float(number))   # error
print(bool(number))    # True
#print(complex(number)) # error
```

```
True
```

In [50]:
```
str1=""
bool(str1)
```

Out[50]:
```
False
```

- complex() : 0j
- int('100.5') : fail
- bool('') : False
- bool(0) : False

In [ ]:
```
int('100')   # 100
int('100.5') # fail
int('python') # fail
int('') # fail

float('100')  # 100.0
float('100.5') # 100.5
float('python')# f
float('')# f

bool('100')  # T
bool('100.5') # T
bool('python')# T
bool('')# F

complex('100')  # 100+0j
complex('100.5') # 100.5+0j
complex('python')# err
complex('')# err
```

In [51]:
```
complex('')
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[51], line 1
----> 1 complex('')

ValueError: complex() arg is a malformed string
```

In [55]:
```
val=100+200j
#int(val)
#float(val)
print(bool(val)) # T
print(str(val))  # '100+200j'
```

```
True
(100+200j)
```

In [56]:
```
str(100)  # '100'
str(100.5) # '100.5'
str(100+200j) # '100+200j'
```

Out[56]:
```
'(100+200j)'
```

In [ ]:
```
val1=True
int(val1)  # 1
float(val1) # 1.0
str(val1)   #"True"
complex(val1) # 1+0j
#### apply all conversion####
val2=False
int(val2) # 0
float(val2) # 0.0
str(val2) # 'False'
complex(val2) # 0j
```

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]: